Q4: MNIST Multi-Layer Perceptron

Parameter count and operations per input image

Architecture. In the supplied Code4 GPU notebook, a 28×28 grayscale image is flattened to 784 inputs. The three fully connected layers are

$$784 \xrightarrow{\texttt{fc1}} 512 \xrightarrow{\texttt{fc2}} 512 \xrightarrow{\texttt{fc3}} 10.$$

Each layer has a bias for every output neuron. The actual forward method applies ReLU after *all three layers*, including the final 10 outputs.

1. Total parameters

A fully connected layer with n_{in} inputs and n_{out} outputs has $n_{\text{in}}n_{\text{out}}$ weights and n_{out} independent bias parameters:

$$P = n_{\text{in}}n_{\text{out}} + n_{\text{out}}.$$

Layer	Weights	Bias parameters	Total parameters
`fc1`: $784 \to 512$	$784(512) = 401{,}408$	512	401,920
`fc2`: $512 \to 512$	$512(512) = 262{,}144$	512	262,656
`fc3`: $512 \to 10$	$512(10) = 5{,}120$	10	5,130
Total	668,672	1,034	669,706

$$\boxed{P_{\text{total}} = 401{,}920 + 262{,}656 + 5{,}130 = 669{,}706 \text{ parameters.}}$$

Flattening and ReLU introduce no trainable parameters.

2. Total operations per image

Each output neuron computes a dot product with n_{in} inputs, giving $n_{\text{in}}n_{\text{out}}$ MACs per layer. Using the specified rules, one MAC costs 2 operations, one bias addition costs 1, and one ReLU costs 1. Therefore, for each layer in this network,

$$O = \underbrace{2n_{\text{in}}n_{\text{out}}}_{\text{MAC operations}} + \underbrace{n_{\text{out}}}_{\text{bias additions}} + \underbrace{n_{\text{out}}}_{\text{ReLU operations}}.$$

Layer	MACs	MAC operations	Bias adds	ReLUs	Total operations
`fc1`	401,408	802,816	512	512	803,840
`fc2`	262,144	524,288	512	512	525,312
`fc3`	5,120	10,240	10	10	10,260
Total	668,672	1,337,344	1,034	1,034	1,339,412

$$\boxed{O_{\text{total}} = 803{,}840 + 525{,}312 + 10{,}260 = 1{,}339{,}412 \text{ operations per image.}}$$

Counting scope. The total covers one forward pass on a preprocessed image. Flattening adds no arithmetic. Training, preprocessing, and device transfers are excluded; GPU execution does not change this count. The final `argmax` has no specified comparison cost. If one comparison costs one operation, selecting the largest of 10 outputs adds $10 - 1 = 9$, giving 1,339,421 operations including class selection.

Verification. Instantiating the notebook's `Net` class and summing `p.numel()` over its parameters gives 669,706. Layer dimensions and the three ReLU calls give the operation totals above.